

AegisHunt 0–100% Implementation Roadmap

Phase 0: 0%–5%

项目定义、架构决策与工程骨架

目标

把模糊题目转化为明确、可实现、可测试的系统需求。

任务

1. 编写系统需求文档。
2. 编写核心使用场景。
3. 编写非功能需求。
4. 创建 Architecture Decision Records:
5. 为什么使用模块化单体;
6. 为什么使用 FastAPI;
7. 为什么使用 Streamlit;
8. 为什么使用 SQLite;
9. 为什么采用监督 + 异常双引擎;
10. 为什么不用 LLM 作为核心检测引擎。
11. 创建项目目录。
12. 配置 pyproject.toml。
13. 配置 pytest、ruff、mypy、pre-commit。
14. 创建 CLI 骨架。
15. 创建 FastAPI 健康检查。
16. 创建 Streamlit 空白页面。
17. 创建 codex_progress.md。

验收

- Python package 可安装。
 - CLI --help 可运行。
 - FastAPI /health 可访问。
 - Streamlit 可打开。
 - pytest、ruff、mypy 通过。
 - 尚未实现模型和网络处理。
-

Phase 1: 5%–12%

配置系统、数据模型与数据库

目标

建立所有后续模块共享的稳定基础。

任务

1. 实现 YAML + environment variables 配置。
2. 使用 Pydantic 定义核心 schema。
3. 使用 SQLAlchemy 定义数据库模型。
4. 初始化 SQLite。
5. 开启 WAL。
6. 建立 repository 层。
7. 建立 schema version。
8. 实现 audit log。
9. 建立数据库初始化 CLI。
10. 编写单元测试。

验收

- 数据库可重复初始化。
 - 核心实体可以写入和读取。
 - 不直接在业务层拼接 SQL。
 - 空数据库可以正常启动 API 和前端。
 - 测试通过。
-

Phase 2: 12%–20%

遥测接入框架

目标

让系统能够接收多种安全数据，而不绑定具体 target。

任务

1. 定义统一 TelemetryIngestor 接口。
2. 实现 PCAP ingestor。
3. 实现 Flow CSV ingestor。
4. 实现 JSON event ingestor。
5. 实现 sample-data ingestor。
6. 建立 ingestion job。
7. 支持进度、状态和错误记录。
8. 计算上传文件 checksum。
9. 实现安全文件存储。
10. 限制上传类型和大小。

11. 创建 sample PCAP 和 sample CSV。
12. 编写 API 和测试。

验收

- 可以上传 PCAP。
 - 可以上传 CSV。
 - 可以查看 ingestion 状态。
 - 错误文件不会导致系统崩溃。
 - E2E 不需要 root。
 - 不要求输入 target。
-

Phase 3: 20%–33%

Packet-to-Flow 与行为特征提取

目标

将原始 PCAP 转换为稳定的双向网络流记录。

任务

1. 解析 IPv4、IPv6、TCP、UDP 和 ICMP。
2. 实现 canonical bidirectional flow key。
3. 第一个数据包方向定义为 forward。
4. 实现 idle timeout。
5. 实现 active timeout。
6. 实现 flow flush。
7. 计算 volume features。
8. 计算 packet-size features。
9. 计算 timing features。
10. 计算 directional features。
11. 计算 TCP flags features。
12. 计算基础 behavioral features。
13. 处理单包流。
14. 处理乱序时间戳。
15. 防止 NaN 和 Infinity。
16. 固定 feature order。
17. 创建 feature dictionary。
18. 编写边界测试。

验收

- 同一双向连接只生成一个 flow。
 - 不同连接不会错误合并。
 - 同一 PCAP 重复处理结果一致。
 - 所有数值合法。
 - feature schema 可导出。
 - 单元测试覆盖核心计算。
-

Phase 4: 33%–42%

数据集注册、转换和质量控制

目标

建立可靠的机器学习数据基础。

任务

1. 建立 dataset registry。
2. 评估候选公开数据集。
3. 选择主 benchmark dataset。
4. 创建下载和转换脚本。
5. 将公开数据映射到统一 schema。
6. 创建 controlled demo dataset。
7. 检测缺失值。
8. 检测重复与近重复。
9. 检测类别不平衡。
10. 检测 label leakage。
11. 检测 group leakage。
12. 使用 capture/session/source-file group split。
13. 生成 train、validation、test。
14. 生成 manifest 和质量报告。
15. 创建 EDA Notebook。

验收

- 数据拆分没有 group overlap。
- 测试集未用于调参。
- 可生成 dataset_manifest。
- 可生成 leakage_report。
- 没有数据时 sample dataset 仍可运行系统。
- 公开数据不可下载时有明确说明，不伪造数据。

Phase 5: 42%–51%

监督式检测引擎

目标

建立已知攻击识别基线 and 主模型。

任务

1. 实现 DummyClassifier。
2. 实现 Logistic Regression。
3. 实现 Decision Tree。
4. 实现 Random Forest。

5. 实现 HistGradientBoosting。
6. 建立 preprocessing pipeline。
7. 使用 GroupKFold。
8. 使用 validation set 调参。
9. 使用 validation set 选择阈值。
10. 实现 probability calibration。
11. 比较模型性能与延迟。
12. 选择主监督模型。
13. 保存 model bundle。
14. 创建 model card。

验收

- 模型选择由真实实验决定。
 - 测试集只用于最终评估。
 - 模型可以独立加载。
 - 特征 schema 不一致时拒绝推理。
 - 所有指标自动计算。
 - 不硬编码最佳模型。
-

Phase 6: 51%–59%

无监督异常检测引擎

目标

检测偏离正常行为基线的活动。

任务

1. 仅使用 benign training data。
2. 实现 Isolation Forest。
3. 标准化 anomaly score。
4. 使用 validation data 选择阈值。
5. 绘制正常和异常 score distribution。
6. 测量 false positive rate。
7. 实现 LOF 离线比较。
8. 可选实现 One-Class SVM 小规模比较。
9. 保存 anomaly model bundle。
10. 创建 anomaly model card。

验收

- 异常模型不使用测试标签训练。
 - anomaly score 有明确范围。
 - 阈值来源可复现。
 - 可以识别 sample anomaly。
 - 正常数据误报率被明确报告。
-

Phase 7: 59%–66%

双引擎融合和未知行为实验

目标

证明双引擎设计是否优于单独模型。

任务

1. 实现 configurable fusion score。
2. 比较 supervised-only。
3. 比较 anomaly-only。
4. 比较 fusion。
5. 实现 Leave-One-Attack-Family-Out。
6. 实现 temporal holdout。
7. 实现 parameter shift。
8. 测量 known attack detection。
9. 测量 unseen-family detection。
10. 评估 fusion 对 recall 和 false positives 的影响。
11. 保存完整实验结果。

验收

- 未见攻击类别不进入训练集。
 - 明确区分已知攻击和未见行为测试。
 - 不使用“保证检测 zero-day”的表达。
 - fusion 权重来自 validation experiment。
 - 结果包含置信区间。
-

Phase 8: 66%–74%

告警、风险评分与可解释性

目标

将模型输出转换为安全分析员可理解的告警。

任务

1. 创建 DetectionResult。
2. 创建统一风险分数。
3. 实现 severity mapping。
4. 创建 SecurityAlert。
5. 实现 reason codes。
6. 实现 global feature importance。
7. 实现 permutation importance。
8. 实现 local feature contribution。
9. 显示 observed value 和 reference range。

10. 明确模型推断与事实的区别。
11. 编写解释性测试。

验收

- 每条告警有证据和原因。
 - 不只显示 “malicious = 1”。
 - 风险分数可配置。
 - feature importance 不被描述为因果关系。
 - 用户可以标记告警 verdict。
-

Phase 9: 74%–81%

Alert Correlation 与 Threat Hypothesis Engine

目标

让项目从 IDS 升级为 threat hunting system。

任务

1. 建立 entity index。
2. 实现 temporal correlation。
3. 实现 source-centered correlation。
4. 实现 destination-centered correlation。
5. 实现 multi-alert escalation。
6. 创建 AlertGroup。
7. 建立 hypothesis templates。
8. 实现 evidence builder。
9. 实现 possible MITRE mapping。
10. 生成 assumptions。
11. 生成 alternative explanations。
12. 生成 investigation queries。
13. 生成 recommended steps。
14. 实现 hypothesis status workflow。

验收

- 多个相关告警可以聚合。
 - 一条孤立低风险告警不一定生成 hypothesis。
 - Hypothesis 包含支持证据。
 - Hypothesis 包含合理的 benign alternatives。
 - Hypothesis 不会自动被标记为 confirmed。
 - 核心功能不依赖 LLM API。
-

Phase 10: 81%–86%

Investigation Case 和 Analyst Feedback

目标

完成威胁狩猎调查闭环。

任务

1. 从 hypothesis 创建 case。
2. 实现 case 状态。
3. 实现 priority。
4. 实现 notes。
5. 实现 evidence attachment/reference。
6. 实现 analyst verdict。
7. 实现 true/false positive feedback。
8. 实现 feedback export。
9. 实现 case report。
10. 创建 retraining candidate dataset。
11. 不自动替换当前模型。

验收

- Case 可以创建、更新和关闭。
 - 所有操作有 audit log。
 - Analyst feedback 可查询。
 - Feedback 可导出。
 - 用户显式触发才可重新训练。
-

Phase 11: 86%–91%

运行时 Pipeline、Replay 与后台任务

目标

让系统能够连续处理数据并实时产生前端结果。

任务

1. 实现完整 runtime pipeline。
2. 实现 PCAP replay。
3. 支持 replay speed。
4. 支持 pause/resume。
5. 支持 progress。
6. 实现 worker status。
7. 实现 graceful shutdown。
8. 实现失败任务恢复。
9. 实现资源监控。

10. 可选实现 live interface capture。
11. Live capture 权限不足时正常降级。

验收

- Replay 可以生成 flows、alerts 和 hypotheses。
 - 不需要 root 即可演示。
 - 中断后数据库保持完整。
 - 前端可读取运行进度。
 - Live capture 失败不会影响 PCAP replay。
-

Phase 12: 91%–96%

FastAPI 和 Streamlit 前端

目标

交付完整、直观的系统演示。

任务

1. 完成全部 API。
2. 完成 Overview。
3. 完成 Data Ingestion。
4. 完成 Traffic Explorer。
5. 完成 Alerts。
6. 完成 Threat Hunts。
7. 完成 Cases。
8. 完成 Model Lab。
9. 完成 Evaluation。
10. 完成 System Health。
11. 实现分页和过滤。
12. 实现错误提示。
13. 实现空状态。
14. 实现自动刷新。
15. 提供 sample demonstration mode。

验收

- 用户可以从前端上传 PCAP。
 - 可以看到处理进度。
 - 可以查看告警解释。
 - 可以查看和管理 hypothesis。
 - 可以创建和更新 case。
 - 可以查看模型实验结果。
 - sample mode 无需额外数据即可演示。
-

Phase 13: 96%–99%

测试、性能、鲁棒性与安全加固

目标

将原型提升为可信、可复现的研究系统。

任务

1. 完成单元测试。
2. 完成集成测试。
3. 完成 E2E。
4. 达到核心模块 80% 覆盖率。
5. 测试损坏 PCAP。
6. 测试错误 CSV schema。
7. 测试超大上传限制。
8. 测试模型 schema mismatch。
9. 测试路径遍历。
10. 测试数据库并发。
11. 测试模型文件损坏。
12. 测量性能。
13. 测量内存。
14. 测量 p50/p95/p99 latency。
15. 完成 robustness experiments。
16. 运行 secret scan。
17. 运行 dependency review。

验收

- pytest 通过。
- ruff 通过。
- mypy 通过。
- E2E 不依赖公网。
- E2E 不依赖管理员权限。
- 错误输入不会导致系统崩溃。
- 不加载任意用户提交的 pickle。
- 上传路径安全。

Phase 14: 99%–100%

部署、文档、演示与论文材料

目标

形成最终可交付成果。

任务

1. 创建 Dockerfile。
2. 创建 Docker Compose。
3. 创建本地安装指南。
4. 创建 macOS 指南。
5. 创建 Linux 指南。
6. 创建 Demo Guide。
7. 创建 3-5 分钟演示脚本。
8. 创建完整 Architecture Document。
9. 创建 Data Model Document。
10. 创建 Feature Dictionary。
11. 创建 Model Cards。
12. 创建 Experiment Protocol。
13. 创建 Limitations。
14. 创建 Threat Model。
15. 生成所有实验图表。
16. 生成所有 CSV 表格。
17. 生成 final experiment summary。
18. 创建 release checklist。
19. 创建 final acceptance report。
20. 运行完整 clean-install test。

最终演示流程

1. 启动 Docker Compose。
2. 打开 Streamlit。
3. 上传 sample PCAP。
4. 启动 replay。
5. 查看 Flow 数据。
6. 查看监督模型结果。
7. 查看 anomaly score。
8. 查看融合风险。
9. 查看生成的 Alerts。
10. 查看 Alert Correlation。
11. 查看 Threat Hypothesis。
12. 将 Hypothesis 转换为 Case。
13. 添加 analyst note。
14. 标记 verdict。
15. 查看模型评估和实验图表。
16. 查看系统性能和健康状态。

最终验收

必须证明完整链路：

PCAP → Packet Parsing → Bidirectional Flows → Behavioral Features → Supervised Detection → Anomaly Detection → Score Fusion → Alerts → Correlation → Threat Hypothesis → Investigation Case → Analyst Feedback → Frontend Visualization

项目只有在上述链路完整可运行时才算 100% 完成。